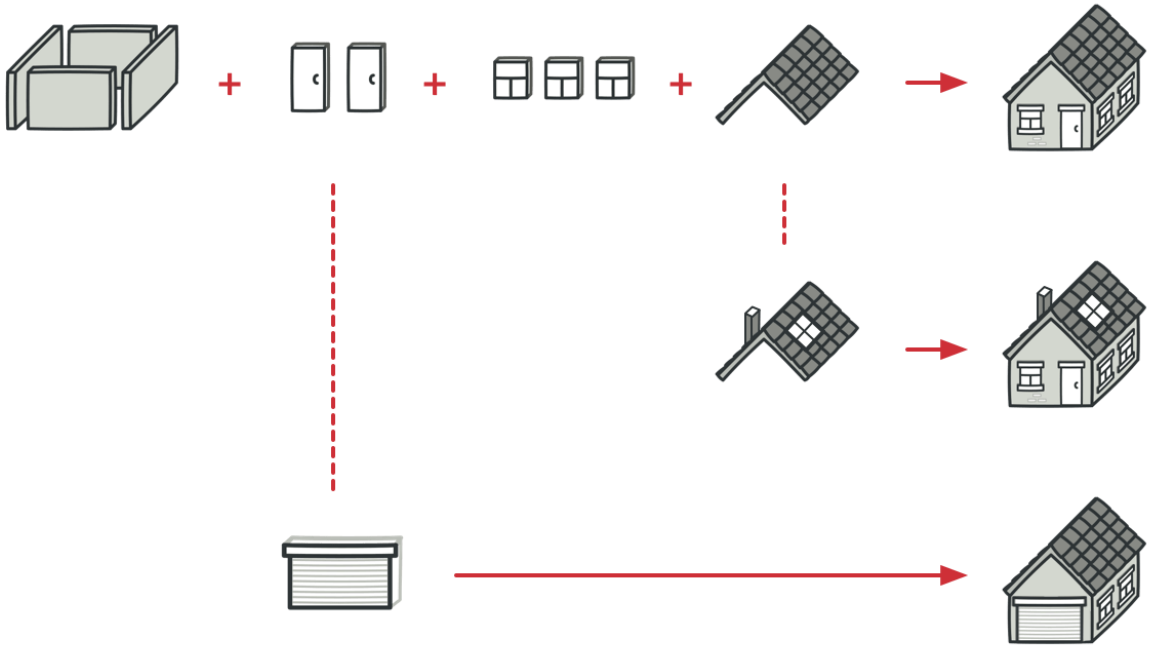
Поведенческие паттерны. Шаблонный метод



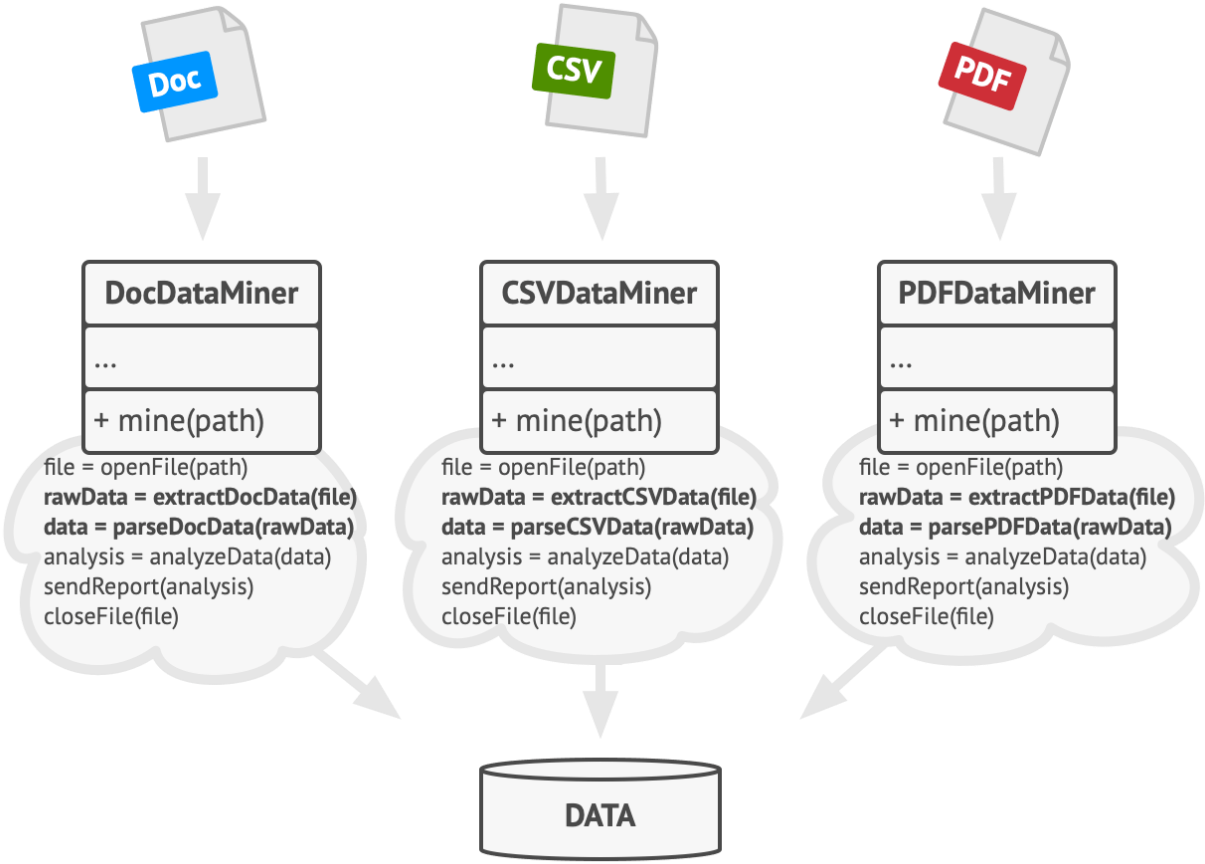
Паттерн Шаблонный метод
Template Method



Паттерн **Шаблонный метод -** формирует структуру алгоритма и позволяет в производных классах реализовать, заместить или переопределить некоторые шаги алгоритма, не изменяя структуру алгоритма в целом.

Проблема

Дублирование кода в сходных по задачам алгоритмах

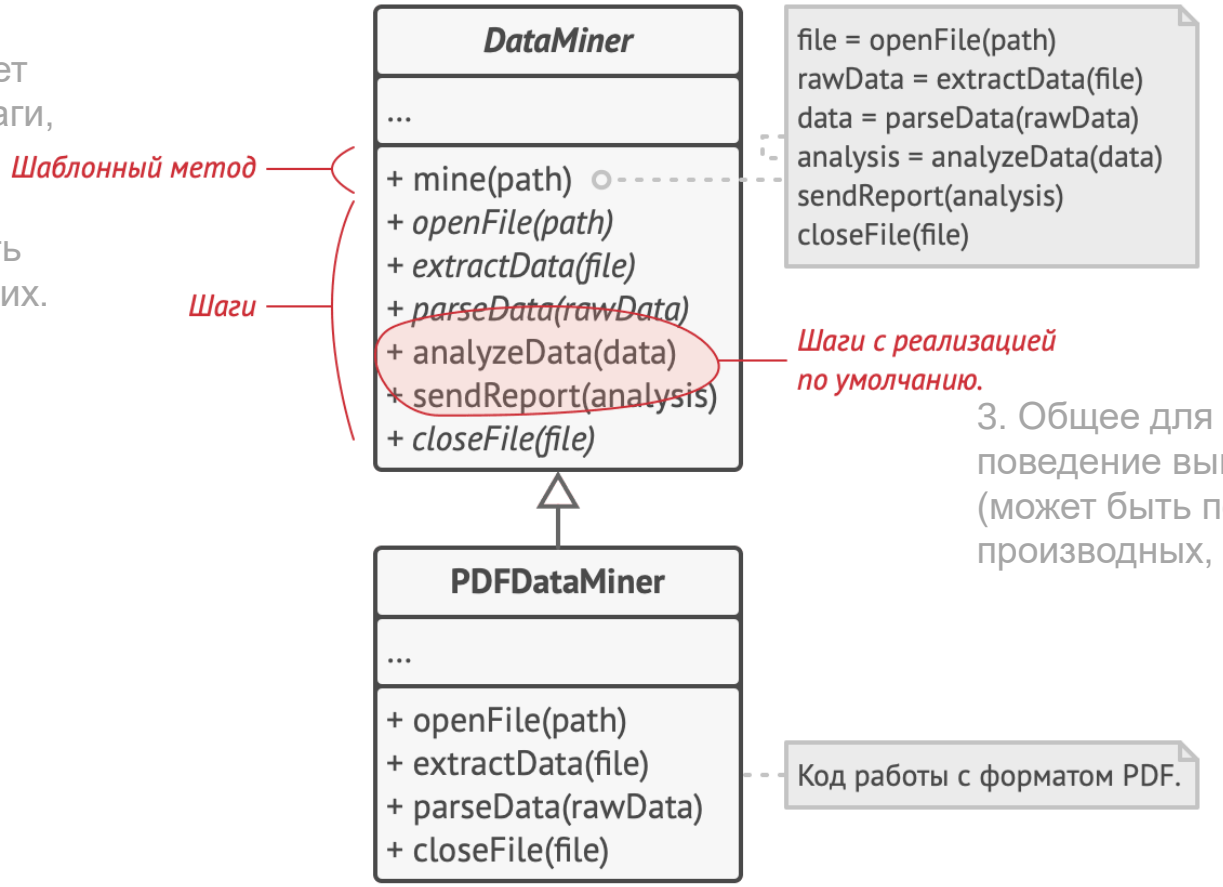


Решение

Паттерн **Шаблонный метод** предлагает разбить алгоритм на последовательность шагов, описать эти шаги в отдельных методах и вызывать их в одном шаблонном методе друг за другом.

Подклассы могут переопределять некоторые шаги алгоритма, оставляя без изменений его структуру и остальные шаги, которые для этого подкласса не так важны.

1. Шаблонный метод разбивает алгоритм на шаги, позволяя подклассам переопределить некоторые из них.



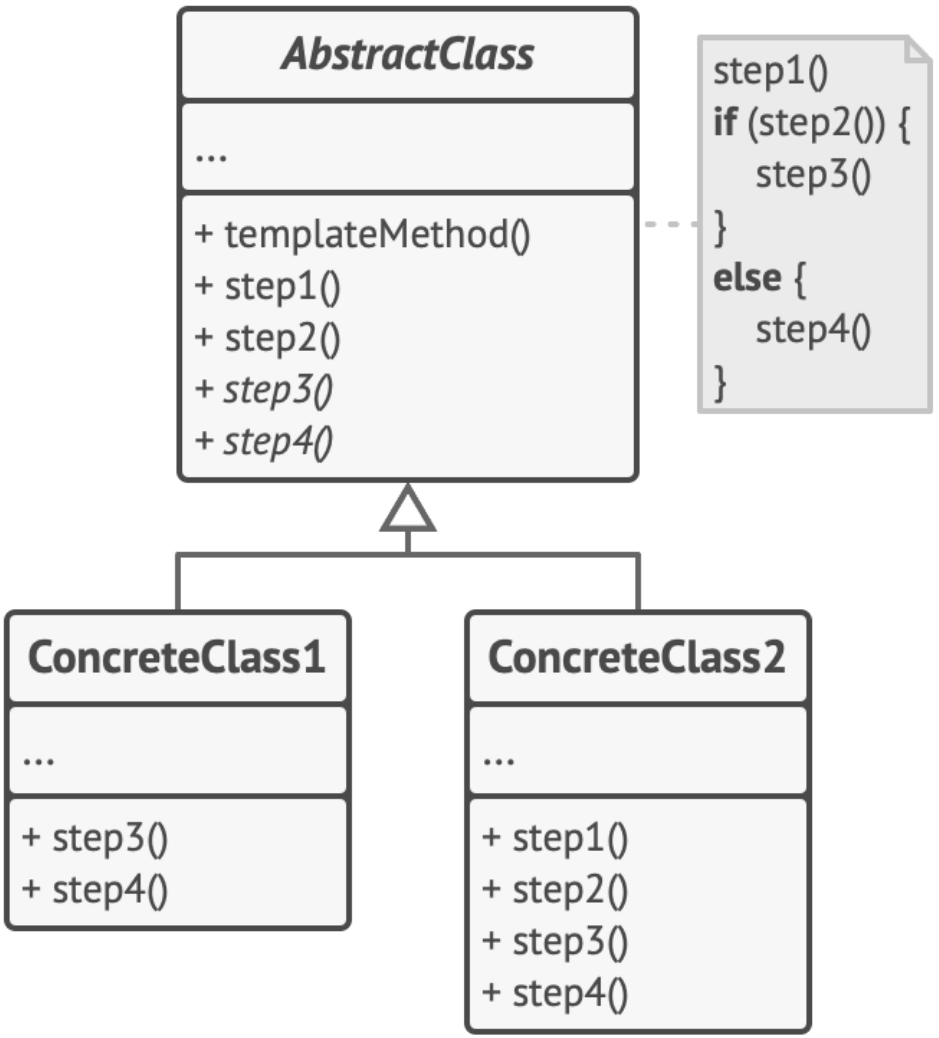
2. Шаги шаблонного метода можно сделать абстрактными. Из-за этого все подклассы должны будут реализовать каждый из шагов по-своему

3. Общее для всех классов поведение вынесено в суперкласс (может быть переопределено в производных, но необязательно)

Структура паттерна

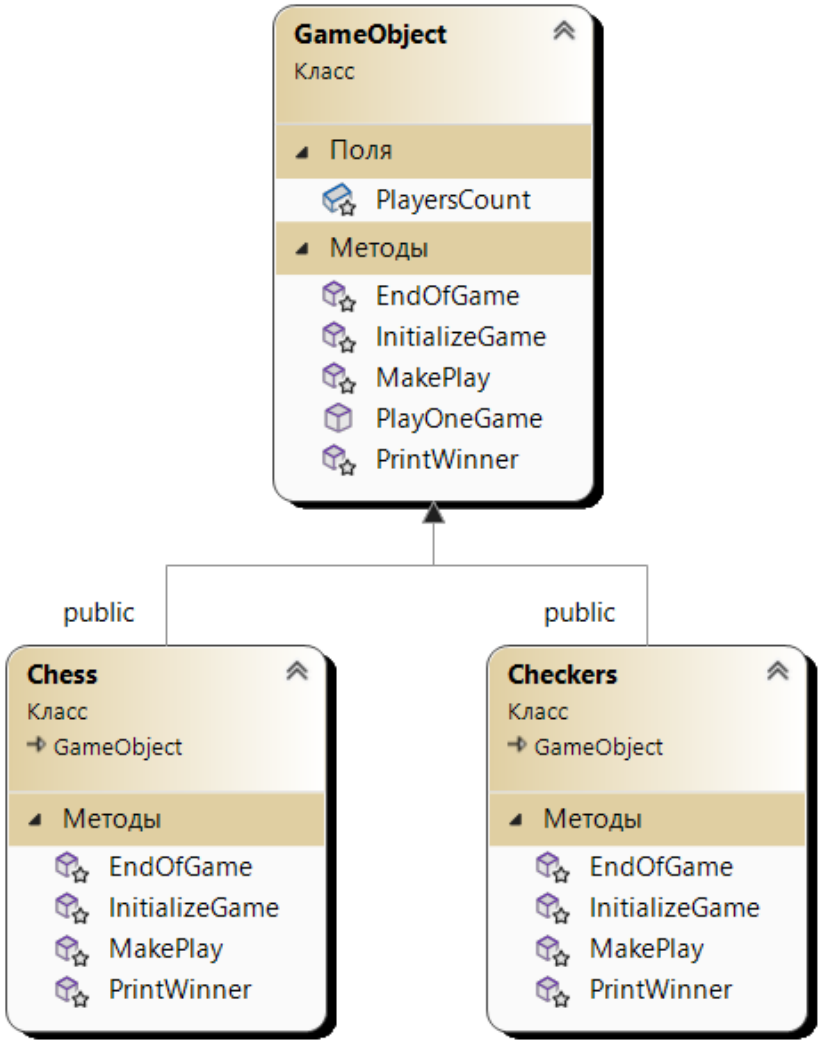
[1]**Абстрактный класс** определяет шаги алгоритма и содержит шаблонный метод, состоящий из вызовов этих шагов. Шаги могут быть как абстрактными, так и содержать реализацию по умолчанию..

[2]**Конкретный класс** переопределяет некоторые (или все) шаги алгоритма. Конкретные классы не переопределяют сам шаблонный метод..



Пример

Применение Шаблонного метода при проектировании игры




```
çľǎşş ĞǎņêÔčkêçţ
 řsộţêçţêđ
    îŋţ RľǎỳêssCộụŋţ

    ŵîsţụǎľ čộộľ ÉŋđÔğĞǎņê
    ŵîsţụǎľ ŵộîđ ÍŋîţîǎľîćêĞǎņê
    ŵîsţụǎľ ŵộîđ ŇǎlêRľǎỳ îŋţ řľǎỳês
    ŵîsţụǎľ ŵộîđ RsîŋţẄîŋŋês
řụčľîç
    ŵộîđ RľǎỳÔŋêĞǎņê îŋţ řľǎỳêssCộụŋţ


çľǎşş Xḥêçlêss   řụčľîç ĞǎņêÔčkêçţ
 řsộţêçţêđ
ŵộîđ ÍŋîţîǎľîćêĞǎņê   ộŵêssîđê

    ŵộîđ ŇǎlêRľǎỳ îŋţ řľǎỳês  ộŵêssîđê

    čộộľ ÉŋđÔğĞǎņê   ộŵêssîđê

    ŵộîđ RsîŋţẄîŋŋês   ộŵêssîđê


çľǎşş Chệşş   řụčľîç ĞǎņêÔčkêçţ
 řsộţêçţêđ

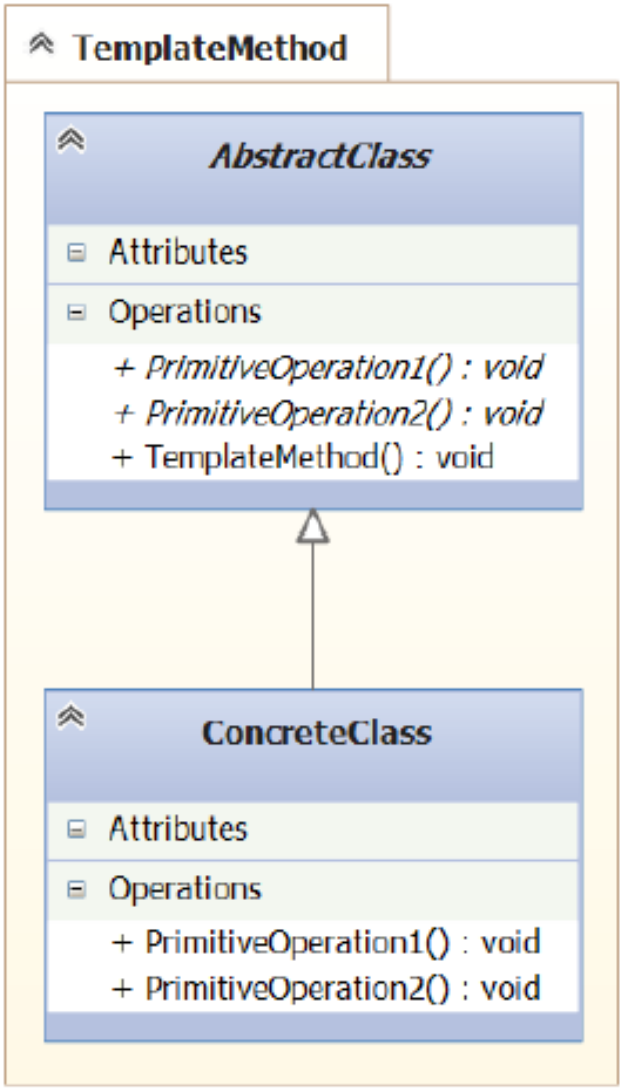
    ŵộîđ ÍŋîţîǎľîćêĞǎņê   ộŵêssîđê

    ŵộîđ ŇǎlêRľǎỳ îŋţ řľǎỳês  ộŵêssîđê

    čộộľ ÉŋđÔğĞǎņê   ộŵêssîđê

    ŵộîđ RsîŋţẄîŋŋês   ộŵêssîđê
```

Структура паттерна UML



Структура паттерна на языке C#

```
class Program
{
    static void Main()
    {
        AbstractClass instance = new ConcreteClass();
        instance.TemplateMethod();
    }
}
```

```
abstract class AbstractClass
{
    public abstract void PrimitiveOperation1();
    public abstract void PrimitiveOperation2();

    public void TemplateMethod()
    {
        PrimitiveOperation1();
        PrimitiveOperation2();
    }
}
```

`AbstractClass` - **Абстрактный класс**:
Содержит абстрактные методы, реализуемые в конкретных производных классах. Каждая реализация отдельного абстрактного метода представляет собой один шаг общего алгоритма.
Абстрактный класс `AbstractClass` содержит реализацию метода `TemplateMethod` в котором вызываются абстрактные методы в определенной последовательности, тем самым формируя структуру общего алгоритма из более мелких алгоритмов представленных методами `PrimitiveOperation1`, `PrimitiveOperation2` … `PrimitiveOperationN`.

```
class ConcreteClass : AbstractClass
{
    public override void PrimitiveOperation1()
    {
        Console.WriteLine("PrimitiveOperation1");
    }

    public override void PrimitiveOperation2()
    {
        Console.WriteLine("PrimitiveOperation2");
    }
}
```

`ConcreteClass` - **Конкретный класс**:
Реализует абстрактные методы `PrimitiveOperation1`, `PrimitiveOperation2` … `PrimitiveOperationN`, из базового класса `AbstractClass`. Реализации абстрактных методов представляют собой отдельные шаги общего алгоритма.

Отношения между участниками

Отношения между классами

- Конкретный класс ConcreteClass связан связью отношения наследования с абстрактным классом AbstractClass.
- Отношения между объектами
- Конкретный класс ConcreteClass предполагает, что инвариантные (постоянные, не изменяющиеся в процессе эволюции системы) шаги алгоритма будут выполнены (через использование техники абстрагирования вариантов использования) в базовом классе AbstractClass.

Применение

1. Когда подклассы должны расширять базовый алгоритм, не меняя его структуры.

Шаблонный метод позволяет подклассам расширять определённые шаги алгоритма используя наследование, не меняя при этом структуру алгоритмов, объявленную в базовом классе.

2. Когда есть несколько классов, делающих одно и то же с незначительными отличиями. Изменениям подвергается один класс, то приходится вносить такие же правки и в остальные классы.

Паттерн шаблонный метод предлагает создать для похожих классов общий суперкласс и оформить в нём главный алгоритм в виде шагов. Отличающиеся шаги можно переопределить в подклассах.

Это позволит убрать дублирование кода в нескольких классах с похожим поведением, но отличающихся в деталях.

void func() final;
void func() final;

Шаги реализации

1. Проанализировать алгоритм с целью разбиения его на шаги. Определить, какие шаги будут стандартными для всех вариаций алгоритма, а какие — изменяющимися.

2. Создать абстрактный базовый класс. Определить в нём шаблонный метод. Этот метод должен состоять из вызовов шагов алгоритма. Имеет смысл сделать шаблонный метод финальным (`void foo() final {}`), чтобы подклассы не могли переопределить его (если язык программирования это позволяет).

3. Добавить в абстрактный класс методы для каждого из шагов алгоритма. Эти методы могут быть абстрактными или иметь какую-то реализацию по умолчанию. В первом случае все подклассы должны будут реализовать эти методы, а во втором — только если реализация шага в подклассе отличается от стандартной версии.

4. Продумать необходимость введения в алгоритм *хуков.** Чаще всего, хуки располагают между основными шагами алгоритма, но можно до и после всех шагов.

5. Создать конкретные классы, унаследовав их от абстрактного класса. Реализовать в них все недостающие шаги и хуки.

**Хуки – пустые методы, которые могут быть переопределены (наполнены) в конкретных классах (необязательно)*

Преимущества и недостатки

+ Облегчает повторное использование кода.

- Жёсткое ограничение структурой существующего алгоритма.

- Может быть нарушен принцип подстановки Барбары Лисков, изменяя базовое поведение одного из шагов алгоритма через подкласс.

- С ростом количества шагов шаблонный метод становится слишком сложно поддерживать.